P3176R0
The Oxford variadic comma

New Proposal, 2024-03-07

This version:
https://eisenwave.github.io/cpp-proposals/oxford-variadic-comma.html

Author:
Jan Schultke<janschultke@gmail.com>

Audience:
SG17

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Source:
eisenwave/cpp-proposals

Abstract

Deprecate ellipsis parameters without a preceding comma. The syntax `(int...)` is incompatible with C, detrimental to C++, and easily replaceable with `(int, ...)`.

Table of Contents

1 Introduction

1.1 History and C compatibility

2 Motivation

2.1 `T.....`

3 Impact on the standard

4 Impact on existing code

5 Design considerations

5.1 Why deprecate, not remove?

5.2 What about `(...)`?

5.3 *parameter-declaration-clause*

6 Proposed wording

6.1 Editorial changes

6.2 Normative changes

7 Acknowledgements

References

Normative References

Informative References

> NOTE: In this proposal, the term *ellipsis parameter* refers to non-template variadic parameters, i.e. "C-style varargs", i.e. as in `(/* */, ...)`.

§ 1. Introduction

[P1219R2] "Homogeneous variadic function parameters" proposed a new feature, where the declarator `(int...)` would be interpreted as a homogeneous function template parameter pack. As part of this, it made it mandatory to separate an ellipsis parameter from preceding parameters using a comma.

The proposal did not pass, but the change to commas achieved strong consensus by EWGI in Kona 2019.

> EWGI Poll: Make vararg declaration comma mandatory.
>
SF	F	N	A	SA
10	6	1	0	0

Going back to 2016, [P0281R0] "Remove comma elision in variadic function declarations" also proposed to make the variadic comma mandatory. Due to its impact on existing code, the proposal did not pass.

My proposal continues where [P1219R2] and [P0281R0] have left off. However, I call for deprecation, not for removal, resulting in the following behavior:

```
   // OK, function template parameter pack
   template<class... Ts> void a(Ts...);
   // OK, abbreviated function template parameter pack
   void b(auto...);
   // OK, ellipsis parameter, compatible with C
   void c(int, ...);
   void d(...);
8  // Deprecated, ellipsis parameter, ill-formed in C
9  void e(int...);
10 void f(int x...);
   // Ill-formed, but unambiguous
   void g(int... args);
```

[P1161R3] "Deprecate uses of the comma operator in subscripting expressions" has followed the same strategy. It deprecated `array[0, 1]` in C++20 which freed up the syntax for [P2128R6] "Multidimensional subscript operator" in C++23.

§ 1.1. History and C compatibility

The active version of the C standard ISO/IEC9899:2018 permits functions which accept a variable number of arguments. Such functions must have a *parameter-type-list* containing at least one parameter, and the ellipsis parameter must be comma-separated:

> *parameter-type-list*:
> *parameter-list*
> *parameter-list* **, ...**

This syntax is unchanged since C89. C has never permitted the comma to be omitted.

Such ellipsis parameters were originally introduced in C++ along with function prototypes, but without a separating comma. Only `int printf(char*...)` would have been well-formed in pre-standard C++, unlike `int printf(char*, ...)`.

For the purpose of C compatibility, C++ later allowed the separating comma, resulting in the syntax (unchanged since C++98):

> *parameter-declaration-clause*:
>
> *parameter-declaration-list*$_{opt}$ `...`$_{opt}$
>
> *parameter-declaration-list* `,` `...`

With the introduction of function template parameter packs in C++11, there arose a syntactical ambiguity for a declarator `(T...)` between:

- a parameter pack of type `T...` and
- a single `T` followed by an ellipsis parameter.

Currently, this ambiguity is resolved in favor of function template parameter packs, if that is well-formed.

§ 2. Motivation

The declarator syntax `(int...)` interferes with future standardization. This has already impacted [P1219R2] and will impact any future proposal which attempts to utilize this syntax.

Furthermore, variadic function templates are a much common and flexible way of writing variadic functions in C++. Many users associate `(int...)` with a pack, not with an ellipsis parameter. Not parsing this syntax as a parameter pack is confusing and has potential for mistakes:

```
template<class Ts>
void f(Ts...); // well-formed, but equivalent to (Ts, ...)
```

Users who are not yet familiar with variadic templates can easily make the mistake of omitting `...` after `class`. Another possible mistake can occur when writing generic lambdas or abbreviated function templates:

```
// abbreviated variadic function template
void g(auto ...args);
// abbreviated non-variadic function template with ellipsis parameter
void g(auto args...);
```

These two forms look awfully similar and are both well-formed with different meaning. The latter should be deprecated.

Lastly, as explained above, only the declarator `(int, ...)` is compatible with C, not `(int...)`. The latter syntax is arguably pointless nowadays and only exists for the sake of compatibility with pre-standard C++.

§ 2.1. `T.....`

C++ also allows `(auto.....)` or `(T.....)`, which is a declarator containing a function template parameter pack, followed by an ellipsis parameter. Such *parameter-declaration-clause*s are confusing because conceptually, they consist of two separate constructs, but the syntax strongly suggests that all `.` apply to `auto` or `T`.

> NOTE: In other words, the declarator `(auto.....)` is equivalent to `(auto..., ...)`.

> NOTE: Some users believe that this construct is entirely useless because one cannot provide any arguments to the ellipsis parameter. However, arguments can be provided if `T...` is template type pack which belongs to the surrounding class template.

§ 3. Impact on the standard

This proposal is a pure deprecation.

No new features are proposed or removed, no semantics are altered, and no existing code is impacted.

§ 4. Impact on existing code

No code becomes ill-formed. Any deprecated uses of `T...` can be converted to `T, ...`. This transformation is simple enough to be performed by automatic tooling.

I conjecture that a non-trivial amount of code would be deprecated. It is hard to find the exact amount because `(T...)` could be using an ellipsis parameter or a parameter pack; one doesn't know without compiling. This prevents simple syntax-based approaches to searching for occurrences. However, I was able to find a few dozen occurrences of `T.....` using a GitHub code search for `/\(.+\.\.\.\.\.\.\).+\{/ language:c++` and other patterns.

§ 5. Design considerations

§ 5.1. Why deprecate, not remove?

I do not propose to remove support for ellipsis parameters without a preceding comma because there is no feature proposed which requires such removal. A deprecation is sufficient at this time.

Sudden removal has already been attempted by [P0281R0] and did not pass. It is unreasonable to suddenly break a large amount of C++ code with little motivation.

The approach of [P1161R3] has more grace.

§ 5.2. What about `(...)`?

The declarator `(...)` should remain valid. It is compatible with C and unambiguous.

§ 5.3. *parameter-declaration-clause*

In the discussion of this proposal, some users have expressed discontent with the current production for *parameter-declaration-clause*. It would be possible to isolate the deprecated case with the following editorial change:

> *parameter-declaration-clause*:
>
> *parameter-declaration-list*$_{opt}$
>
> *parameter-declaration-list* **...**
>
> *parameter-declaration-list* **, ...**
>
> **...**

Of these productions, the second (highlighted) is deprecated. If the variadic comma was made mandatory, this rule would simply be removed and the rest left intact.

I believe that this editorial change is beneficial to the proposal and improves readability of the *parameter-declaration-clause* production in general.

§ 6. Proposed wording

The proposed wording is relative to [N4950].

§ 6.1. Editorial changes

In subclause 9.3.4.6 [dcl.fct], modify paragraph 3 as follows:

> *parameter-declaration-clause*:
> ~~*parameter-declaration-list*$_{opt}$ **...**$_{opt}$~~
> *parameter-declaration-list*$_{opt}$
> *parameter-declaration-list* **...**
> *parameter-declaration-list* **, ...**
> **...**

Update Annex A, subclause 7 [gram.dcl] accordingly.

§ 6.2. Normative changes

In subclause 9.3.4.6 [dcl.fct], add a paragraph:

> A *parameter-declaration-clause* of the form *parameter-declaration-list* **...** is deprecated.

In Annex D [depr], add a subclause:

D.X Non-comma-separated ellipsis parameters [depr.ellipsis.comma]

A *parameter-declaration-clause* of the form *parameter-declaration-list* `...` is deprecated.
[*Example*:

```
void a(...);                // OK
void b(auto...);            // OK
void c(auto, ...);          // OK

void d_depr(auto......);    // deprecated
void d_okay(auto..., ...);  // OK

void e_depr(auto x...);     // deprecated
void d_okay(auto x, ...);   // OK

void f_depr(auto...x...);   // deprecated
void f_okay(auto...x, ...); // OK

void g_depr(int...);        // deprecated
void g_okay(int, ...);      // OK

void h_depr(int x...);      // deprecated
void h_okay(int x, ...);    // OK
```

— *end example*]

§ 7. Acknowledgements

I thank Christof Meerwald for contributing proposed wording.

I thank Arthur O'Dwyer for providing extensive editorial feedback, including the suggestion to update *parameter-declaration-clause*.

§ References

§ Normative References

[N4950]

Thomas Köppe. *Working Draft, Standard for Programming Language C++*. 10 May 2023. URL: https://wg21.link/n4950

§ Informative References

[P0281R0]

James Touton. *Remove comma elision in variadic function declarations*. 23 January 2016. URL: https://wg21.link/p0281r0

[P1161R3]

Corentin Jabot. *Deprecate uses of the comma operator in subscripting expressions*. 22 February 2019. URL: https://wg21.link/p1161r3

[P1219R2]

James Touton. *Homogeneous variadic function parameters*. 6 October 2019. URL: https://wg21.link/p1219r2

[P2128R6]

Corentin Jabot, Isabella Muerte, Daisy Hollman, Christian Trott, Mark Hoemmen. *Multidimensional subscript operator*. 14 September 2021. URL: https://wg21.link/p2128r6